Project Proposal

Project: Smart Clinic — an AI assistant that guides patients before they visit the doctor

Objective

Help patients with chest and respiratory symptoms understand their situation and reach the right specialty faster. The patient describes their symptoms, uploads a chest X-ray and its report, and receives one clear summary in simple language. This reduces unnecessary worry, wrong-specialty visits, and the time spent in the first examination.

Description

Smart Clinic is a website that works as a smart assistant before the doctor's visit. The patient does three simple things: writes their symptoms in their own words, uploads a chest X-ray image, and uploads or types the X-ray report. The website then shows a single summary page containing:

- The possible diseases based on the symptoms.
- A preliminary opinion on the X-ray image (normal or pneumonia) with a confidence value.
- A simplified explanation of the X-ray report, with key medical terms explained.
- The suggested specialty to visit, and a downloadable summary to show the doctor.

The system is built from three AI components: Symptoms (Machine Learning), X-ray (Deep Learning + Computer Vision), and Reports (NLP), plus an Arabic layer that converts the patient's own phrases into symptom names.

Medical disclaimer: The output is a preliminary guide, not a final diagnosis and not a substitute for a doctor. This notice appears on every page.

Tools & Design

- **Data:** Kaggle datasets — Disease Prediction Using Machine Learning (4,920 cases, 41 diseases, 132 symptoms), Chest X-Ray Images (Pneumonia) (5,863 images), IU X-Ray / Open-i (3,955 reports).
- **Workflow design:** 5-step user journey: Write symptoms → Possible diseases → Upload chest X-ray → Read X-ray report → Final summary page.
- **Website design:** Four pages (symptoms, X-ray upload, report, final summary) connected to the three models through an API, with the medical disclaimer on every page.
- **Development environment:** Google Colab / Kaggle Notebooks (free GPU), GitHub for code, Visual Studio Code.
- **Experiment tracking:** MLflow, set up from day one with an agreed naming convention.

Technologies to Use

- **Languages & data:** Python, Pandas, NumPy, Matplotlib, Seaborn.
- **Machine Learning:** Scikit-learn — Logistic Regression, Decision Tree, Naive Bayes, KNN, SVM, Random Forest, Boosting, PCA / Feature Selection.
- **Deep Learning & Computer Vision:** TensorFlow / Keras, CNN from scratch, Transfer Learning (ResNet / VGG), Dropout, Early Stopping.
- **NLP:** NLTK / text cleaning, TF-IDF, Hugging Face Transformers, Regex for the Arabic symptom dictionary.

- **Web & integration:** Streamlit, REST API.
- **Cloud & MLOps:** MLflow, Azure Machine Learning, Azure deployment, GitHub.

Team Members & Responsibilities

1- **Member 1 – Data:** downloads and cleans the three datasets, produces plots and exploratory analysis, and splits the data (80/10/10).
2- **Member 2 – Symptoms Model (ML):** trains and compares symptom-to-disease models, feature reduction, Random Forest and Boosting.
3- **Member 3 – X-ray Model (CNN):** prepares the images and builds the CNN from scratch.
4- **Member 4 – X-ray Model (Transfer Learning):** builds the Transfer Learning model, compares versions, and analyses X-ray errors.
5- **Member 5 – Reports Model (NLP):** cleans and classifies reports, Hugging Face model, and the Arabic symptom dictionary.
6- **Member 6 – Website & Cloud:** Streamlit website, connecting the three models, MLflow, Azure, and publishing.

Shared by all members: record experiments in MLflow, write a paragraph about their part in the final report, attend a short weekly meeting, and rehearse the final presentation.

Week 1: Setup, Data Preparation & First Baselines

- **All members:** Kick-off meeting to agree on work division, file naming, and a shared GitHub repository; create Kaggle, GitHub, and Azure accounts; prepare Colab / Kaggle Notebooks.
- **Member 6:** Create the project structure (a folder per part) and set up MLflow before any training.
- **Member 1:** Download, check (missing values, duplicates), plot, and split the three datasets; save clean copies in the shared location.
- **Members 2, 3, 4, 5:** Read the data and run a first simple model on it (Logistic Regression baseline, simple CNN, TF-IDF classifier), even if accuracy is weak.

Deliverables:

- GitHub repository with project structure and README.
- MLflow tracking server running with an agreed experiment naming method.
- Clean, split datasets (symptoms, X-rays, reports) and an exploratory plots report.
- A first baseline model for each part, logged in MLflow.

Week 2: Model Development & Comparison

- **Member 2:** Try Decision Tree, Naive Bayes, KNN, SVM; evaluate with Accuracy, Precision, Recall, F1; analyse confused diseases; try feature selection and PCA.
- **Member 3:** Improve the CNN from scratch with Dropout and Early Stopping; measure results.
- **Member 4:** Build the Transfer Learning model (ResNet / VGG) and compare it with the CNN.
- **Member 5:** Clean report text, apply TF-IDF, train the normal/abnormal classifier, and test a Hugging Face model.
- **Member 1:** Support the team with data fixes and visualisations of model errors.
- **Member 6:** Design the website pages and prepare the Streamlit skeleton.

Deliverables:

- Comparison table of the symptom models with error analysis.
- CNN and Transfer Learning X-ray models with a comparison table.
- TF-IDF report classifier and Hugging Face comparison.
- All experiments recorded in MLflow.

Week 3: Best Models, Arabic Layer & Website

- **Member 2:** Train Random Forest and Boosting; choose and save the best symptom model.
- **Members 3 & 4:** Choose and save the best X-ray model; analyse the images where the model made mistakes.
- **Member 5:** Select the best report model; build the Arabic dictionary and Regex rules that map patient phrases to symptom names.
- **Member 6:** Build the website pages (symptoms, X-ray upload, report, final summary) and add the medical disclaimer on every page.
- **Member 1:** Prepare test cases and sample inputs for the end-to-end flow.

Deliverables:

- Best saved model for each of the three parts.
- Arabic symptom dictionary with extraction rules.
- Website pages built and running locally.
- Updated MLflow log with final model choices and reasons.

Week 4: Integration, Cloud Deployment & Final Delivery

- **Member 6:** Connect the three models to the website and combine results into one downloadable summary page; upload models to Azure Machine Learning and publish the website online.
- **Members 2, 3, 4, 5:** Support integration, fix issues in their models, and test with real examples.
- **Member 1:** Run end-to-end testing and compile the plots and data report.
- **All members:** Write the final report, prepare the presentation, and rehearse the live demo (each member can explain any part).

Deliverables:

- Smart Clinic website running on the internet.
- Three final models: symptoms, X-ray, and reports.
- Comparison table of models tried and reasons for each choice, plus complete MLflow experiment log.
- Final report, presentation, and live demo.
- Organised GitHub code with instructions on how to run it.